

CSE 719: Distributed & Cloud Computing

Topic 7 — Distributed System Fundamentals

Complete Model Answers (2020–2022)

University of Chittagong — 8th Semester B.Sc. (Engg.) Examination

Source: Lecture-01 (Distributed Systems intro) + standard DS textbook

Contents

1	Core Theory — Reusable Across Years	2
1.1	Defining a Distributed System	2
1.2	The 9 Design Goals (Lecture-01)	2
1.3	Key “Hard” Issues of Distributed Systems	2
2	2020 Examination (CSE-813)	3
2.1	Q1(a) — Define distributed and cloud systems; give examples [2+1]	3
2.2	Q1(b) — Tightly vs loosely coupled systems [3.25]	3
2.3	Q1(c) — Distributed vs parallel processing [2.5]	3
2.4	Q2(a) — Location vs relocation vs migration transparency [3]	4
2.5	Q2(c) — Design principles for better performance [3]	4
3	2021 Examination (CSE 813)	5
3.1	Q1(a) — Define a distributed system; real-time examples [1.75]	5
3.2	Q1(b) — Advantages over standalone; disadvantages [2.5]	5
3.3	Q1(c) — Scalability + challenges [2.5]	6
3.4	Q1(d) — Marshalling/Unmarshalling; ACID [2]	6
4	Quick Summary — Exam Patterns	7

Core Theory — Reusable Across Years

Defining a Distributed System

Tanenbaum: “A distributed system is a collection of independent computers that appear to its users as a single coherent system.”

Working definition (Lecture-01): a collection of **entities**, each **autonomous**, **programmable**, **asynchronous** and **failure-prone**, that communicate through an **unreliable communication medium**.

- **Entity** = a process on a device (PC, PDA, server).
- **Communication medium** = wired or wireless network.

Examples: Client–Server (NFS), the Web, the Internet, DNS, P2P overlays (BitTorrent), a cloud (Amazon EC2/S3, Azure), a datacenter (Google).

The 9 Design Goals (Lecture-01)

Goal	Question it answers
Heterogeneity	Can it handle many types of PCs/devices?
Robustness	Is it resilient to host crashes, failures, dropped messages?
Availability	Are data + services always there for clients?
Transparency	Can it hide internal workings from users?
Concurrency	Can the server handle many clients at once?
Efficiency	Is the service fast; does it use resources fully?
Scalability	Can it grow to millions of nodes without degrading?
Security	Can it withstand attacks?
Openness	Is the system extensible?

Key “Hard” Issues of Distributed Systems

- **No global clock** — no single correct notion of time (**asynchrony**).
- **Unpredictable failures** — a missing response may be a crashed node, a down network path, or a slow link (**failure-prone**, **unreliable**).
- **Highly variable bandwidth** (Kbps → Tbps) and **variable latency** (ms → seconds).
- **Large, variable scale** — from 2 to several million hosts.

2020 Examination (CSE-813)

Q1(a) — Define distributed and cloud systems; give examples [2+1]

Define distributed and cloud systems. Give appropriate examples.

Distributed system: a collection of independent computers that appears to users as a single coherent system; entities are autonomous, asynchronous and failure-prone, communicating over an unreliable network. *Examples:* the Web, NFS, DNS, BitTorrent.

Cloud system: a model for **on-demand, network access to a shared pool of configurable computing resources** (servers, storage, applications) that are rapidly provisioned and released, billed by consumption. *Examples:* Amazon EC2/S3, Microsoft Azure, Google Cloud.

Link: a cloud is a large-scale distributed system offered as a metered utility.

Q1(b) — Tightly vs loosely coupled systems [3.25]

Differentiate tightly and loosely coupled systems with appropriate figures.

	Tightly coupled	Loosely coupled
Memory	Shared common memory (multiprocessor)	Private local memory each node (multicomputer)
Communication	Through shared memory	By message passing over network
Clock	Often common clock	Independent clocks
Coupling	High; fast, low-latency	Low; slower, scalable
Example	SMP / parallel machine	Distributed system / cluster

Figure (describe in exam):

- Tightly coupled: several CPUs → one shared memory block via a common bus.
- Loosely coupled: each CPU has its *own* memory; CPUs connected by a network/communication link.

Q1(c) — Distributed vs parallel processing [2.5]

In what respect are distributed computing systems better than parallel processing systems?

- **Scalability** — distributed systems scale to geographically dispersed, virtually unlimited nodes; parallel systems are limited by a single machine's bus/memory.
- **Resource & information sharing** — remote resources (files, printers, data) shared across sites; parallel systems share only local resources.
- **Reliability / fault tolerance** — no single point of failure; a crashed node need not stop the system. Parallel systems share memory, so a fault is more likely to be fatal.
- **Cost-effectiveness** — built from commodity machines vs. expensive tightly-coupled hardware.
- **Geographic distribution** — services located near users (lower latency); parallel processing is co-located.

Q2(a) — Location vs relocation vs migration transparency [3]

How are location, relocation, and migration transparencies different from each other in distributed computing? Explain with examples.

Type	Hides	Example
Location	<i>Where</i> a resource is physically located	A URL/DNS name gives no hint of the server's physical site.
Relocation	That a resource <i>moved</i> to another place while in use	A mobile user keeps a connection while the service is moved between servers.
Migration	That a resource has <i>moved</i> to another location (between accesses)	A file moved to another server; clients still use the same name.
Key distinction: migration = resource can move (name unchanged); relocation = resource moves <i>even while being accessed</i> ; location = you cannot tell where it is at all.		

Q2(c) — Design principles for better performance [3]

What are the design principles considered useful for better performance in a distributed system?

- **Minimise network traffic / communication** — fewer, batched messages; exploit locality.
- **Caching & replication** — keep data near where it is used; reduce remote access.
- **Exploit parallelism / concurrency** — overlap computation and communication.
- **Reduce remote-access overhead** — move computation to data when cheaper.
- **Load balancing** — distribute work evenly across nodes.
- **Avoid centralised bottlenecks** — no single server/coordinator that all requests pass through.

2021 Examination (CSE 813)

Q1(a) — Define a distributed system; real-time examples [1.75]

Define a distributed system. Mention a few real-time examples of distributed systems.

Definition: a collection of independent computers that appears to its users as a single coherent system, communicating by message passing over an unreliable network.

Real-time examples: air-traffic control, ATM / banking networks, telecom switching, multiplayer online games, sensor networks, the World Wide Web, Google search infrastructure.

Q1(b) — Advantages over standalone; disadvantages [2.5]

Mention the advantages of a distributed computing environment over standalone applications. What are the disadvantages of distributed systems?

Advantages:

- Resource & information sharing across sites.
- Higher reliability / fault tolerance (no single point of failure).
- Better price/performance via commodity hardware.
- Scalability and incremental growth.
- Geographic distribution $\Rightarrow$ lower latency to users.

Disadvantages:

- **Complexity** — harder to design, debug, and reason about (no global clock).
- **Network dependence** — failures/congestion degrade or stop the system.
- **Security** — more attack surface; data crosses public links.
- **Concurrency & consistency** problems from shared data.
- Troubleshooting and management overhead.

Q1(c) — Scalability + challenges

[2.5]

Define scalability. What are the challenges involved in designing scalable distributed systems?

Scalability: the ability of a system to handle a growing amount of work (more nodes, users, or geographic spread) **without significant loss of performance**. Three dimensions: **size, geographic, administrative**.

Challenges:

- **Centralised components** (single server), **centralised data** (single DB), **centralised algorithms** (global state) all become bottlenecks $\Rightarrow$ avoid them.
- **Communication latency** grows over wide-area distances.
- **Keeping replicas consistent** as the system grows.
- **Multiple administrative domains** — conflicting policies, security, trust.

Techniques: hiding latency (async/caching), distribution (partitioning), replication.

Q1(d) — Marshalling/Unmarshalling; ACID

[2]

Define Marshalling and Unmarshalling. What are the ACID properties?

Marshalling: packing a collection of data items into a form (a flat byte stream / external representation) suitable for transmission across a network — i.e. **serialization** of parameters/objects before sending.

Unmarshalling: the inverse — **disassembling** the received stream back into the original data structures at the destination.

ACID properties (transactions):

Atomicity	All-or-nothing: a transaction either completes fully or has no effect.
Consistency	Takes the database from one valid state to another (invariants preserved).
Isolation	Concurrent transactions do not interfere; result equals some serial order.
Durability	Once committed, effects survive crashes (persisted).

Quick Summary — Exam Patterns

- **Q1 of Section A is almost always fundamentals** (2020 & 2021). Have the **definition** + **examples** ready cold — it opens both papers.
- **Definition** (Tanenbaum one-liner + working definition) and **examples** recur every year — guaranteed easy marks.
- **Transparencies** (2020 Q2a): know all named ones but be ready to *differentiate* location / relocation / migration specifically.
- **Coupling** (2020 Q1b): always asks for **figures** — draw shared-memory vs private-memory diagrams.
- **Scalability** (2021 Q1c): define + the 3 centralisation pitfalls.
- **Marshalling + ACID** (2021 Q1d): two-part; keep each crisp — serialization + the 4 ACID terms.
- *Note:* only Lecture-01's definition/design-goals/issues are slide-backed; coupling, transparencies, ACID come from the textbook — confirm figures from Sinha before the exam.